KLE Technological University | Creating Value, Leveraging Knowledge

Department of Computer Science and Engineering (Artificial Intelligence)

FMTH0306-1.0

Course Plan

Semester: V Year: 2026-27

Course Title: Parallel and GPU Computing	Course Code: 26ECAC304	
Total Contact hrs: 70 hrs	Duration of ESA: 3 hrs	
ISA Marks: 50	ESA Marks: 50	Total Marks: 100
Lesson Plan Authors: Ms Shrushti Gani, Mr Dheeraj Kumar	Date: 18-08-2026	
Checked By: Dr.Narayan D G	Date: 03-09-2026	

Prerequisites: Basic knowledge of Computer Organization, Operating Systems, and Programming

Course Outcomes (COs):

At the end of the course, the student should be able to:

1. **Describe** the fundamental concepts and principles of parallel computing.
2. **Analyze** different parallel computing systems, architectures, and memory organization techniques.
3. **Apply** parallel programming models and techniques to develop efficient solutions for computational problems.
4. **Apply** GPU programming and optimization techniques to improve the performance of parallel applications.
5. **Optimize** parallelism techniques for scalable and high-performance AI models.

Course Articulation Matrix: Mapping of Course Outcomes (COs) with Program Outcomes (POs)

Course Title: Parallel and GPU Computing												Semester: V	
Course Code: 26ECAC304												Year: 2026-27	
Course Outcomes (COs) / Program Outcomes (POs)	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**
1. Explain the fundamentals of parallel computing, concurrency and parallelism, Flynn's taxonomy, and analyze speedup using Amdahl's Law and Gustafson's Law.	H											M	
2. Analyze multicore architecture, shared and distributed memory systems, cache hierarchy, cache coherence, memory consistency,and synchronization mechanisms in NUMA systems.	H	M			H								
3. Apply parallel programming models and frameworks such as pthreads, OpenMP, and Intel TBB for data/task parallelism, work partitioning, and scheduling.			H		H							M	
4. Apply GPU architecture and the CUDA programming model to develop and optimize GPU kernels using the SIMT execution model and GPU memory hierarchy.	M		H		H							H	
5. Optimize parallelism techniques in scalable AI models, including data/model/pipeline parallelism, GPU clusters, distributed training, MapReduce, and performance profiling and optimization.			M		H							H	

Degree of compliance **L**: Low **M**: Medium **H**: High

Competency addressed in the Course, corresponding Performance Indicators (PI) and Course Specific Performance Indicators (CSPI)

Competency	Performance Indicators(PI)	Course Specific Performance Indicators (CSPI)
1.4: Demonstrate the competency in Computer Science and Engineering (WK4)	1.4.1:Apply systems engineering principles to design hardware and software solutions, adopt lifecycle models, and evaluate system performance, scalability, and reliability.	1.4.1.1: Explain the motivation for parallel computing and the limitations of single-core processors.
		1.4.1.2: Apply Flynn's taxonomy and models of parallel computation to classify parallel systems.
		1.4.1.3: Demonstrate understanding of multicore CPU architecture, memory hierarchy, and cache organization.
2.1: Demonstrate an ability to identify and define complex engineering problems by understanding real-world contexts and constraints.	2.1.2: Identify critical systems/sub systems, variables, and constraints that characterize the problem environment by analyzing practical contexts and real-world limitations.	2.1.2.1: Identify cache coherence and memory consistency issues in multicore systems.
		2.1.2.2: Analyze synchronization primitives such as locks, semaphores, and barriers to prevent race conditions and deadlocks.
		2.1.2.3: Optimize memory access patterns to reduce false sharing and NUMA-related latency.
3.2/3.3: Demonstrate an ability to generate a diverse set of creative alternative design solutions, and to select an optimal design solution.	3.3.2: Build models/prototypes to develop chosen design solutions.	3.3.2.1: Develop multithreaded and OpenMP-based programs applying data and task parallelism.
		3.3.2.2: Develop CUDA kernels applying the thread hierarchy and SIMT execution model.
		3.3.2.3: Optimize GPU memory usage by comparing global and shared memory access patterns.
	3.2.2: Identify suitable criteria for the evaluation of alternate design solutions.	3.2.2.1: Evaluate CPU vs GPU performance for AI workload inference.
		3.2.2.2: Profile parallel programs to identify compute-bound and memory-bound bottlenecks.
5.2: Demonstrate an ability to apply discipline-specific tools, techniques, and resources.	5.2.1: Create/modify parts and models using modern engineering and IT tools for the solution of complex engineering problems.	5.2.1.1: Apply data, model, and pipeline parallelism for distributed AI training.
		5.2.1.2: Apply the MapReduce programming model and GPU-based acceleration for large-scale data processing.
		5.2.1.3: Apply profiling and optimization techniques to accelerate deep learning training and inference.
12.2: Ability to deploy reliable software systems through systematic testing and validation.	12.2.2: Deploy a reliable software system by identifying appropriate resources.	12.2.2.1: Deploy and evaluate GPU-accelerated computations, comparing CPU vs GPU performance.
		12.2.2.2: Evaluate scalable AI/data-intensive workloads considering throughput, latency, scalability, and resource utilization.
		12.2.2.3: Deploy distributed training workflows using GPU clusters with optimized communication and synchronization.

Eg: 1.2.3.4: Represents program outcome '1', competency '2', performance indicator '3 and **Course** Specific Performance Indicator '4

Course Content

Course Code: 26ECAC304	Course Title: Parallel and GPU Computing	
L-T-P: 3-0-1	Credits: 4	Contact Hrs: 3 hrs/week
ISA Marks: 50	ESA Marks: 100	Total Marks: 100
Teaching Hrs: 40	Practical Hrs: 30hrs	Exam Duration: 3 hrs

Content	Hrs.
Unit - I	
Chapter 1:Introduction: Motivation for parallel computing, limitations of single-core processors, concepts of concurrency and parallelism, Flynn's taxonomy, models of parallel computation, Amdahl's Law and Gustafson's Law, evolution of multicore processors, architecture of modern multicore CPUs, shared and distributed memory architectures, memory hierarchy, cache organization and performance considerations.	7 hrs
Chapter 2:Memory Systems and Synchronization Cache:coherence protocols, memory consistency models, NUMA architecture and memory access characteristics, shared memory communication mechanisms, synchronization primitives including locks, semaphores, barriers, and atomic operations, false sharing and its impact on performance, race conditions and deadlocks, performance challenges and memory access optimization.	8 hrs
Unit – II	
Chapter 3: Parallel Programming Models and Patterns Thread-based programming models, data parallelism and task parallelism, shared-memory programming concepts, parallel programming frameworks including pthreads, OpenMP, and Intel TBB, work partitioning strategies, task scheduling techniques, parallel algorithm design patterns including reduction, map, pipeline, and divide-and-conquer approaches.	8 hrs
Chapter 4: GPU Architectures and Many-Core Systems :GPU architectures and many-core systems, GPU execution model and thread hierarchy, SIMT execution model, CUDA programming model, GPU memory hierarchy, kernel execution and optimization techniques, GPU-based parallel processing for data-intensive applications, acceleration of machine learning and deep learning workloads, CUDA-based optimization techniques including memory coalescing, thread scheduling, and performance tuning for AI workloads.	7 hrs
Unit – III	
Chapter 5: Parallelism in AI : Systems Overview of AI workloads in modern AI data centers, data parallelism, model parallelism and pipeline parallelism, GPU clusters and distributed training fundamentals, communication overheads and synchronization issues, scalability, load balancing and performance trade-offs.	5 hrs
Chapter 6: Scalable AI Systems: MapReduce programming model and workflow, GPU-based acceleration for large-scale data processing, introduction to distributed processing frameworks, profiling and bottleneck analysis including compute-bound and memory-bound workloads, optimization techniques including memory and kernel optimization, case study of deep learning training and inference acceleration.	5 hrs

Text Books:

1. W.-M. W. Hwu, D. B. Kirk, and I. El Hajj, "Programming Massively Parallel Processors: A Hands-on Approach", 4th ed., Morgan Kaufmann, 2023.
2. G. Barlas, "Multicore and GPU Programming: An Integrated Approach", 2nd ed., Morgan Kaufmann, 2022.

Reference Books:

1. M. Herlihy and N. Shavit, "The Art of Multiprocessor Programming", 2nd ed., Morgan Kaufmann, 2020.
2. S. Mittal, "CUDA Programming from Basics to Advanced", 1st ed., CRC Press, 2024.

Evaluation Scheme

ISA Scheme

Assessment	Conducted for marks	Weightage in Marks
ISA-1 (Theory)	40	33
ISA-2 (Theory)	40	
Lab Activity	20	17
Total		**50**

ESA Scheme

Assessment	Conducted for marks	Weightage in Marks
Theory	**100**	**33**
Lab	**50**	**17**
Total		**50**

Course Unitization for ISA and ESA

Topics / Chapters	Teaching Credits	No. of Questions in ISA-1	No. of Questions in ISA-2	No. of Questions in Activity	No. of Questions in ESA
Unit I					
Chapter 1: Introduction.	07 hrs	1	--	0	1
Chapter 2: Memory Systems and Synchronization.	08 hrs	1	--	1	1
Unit II					
Chapter 3: Parallel Programming Models and Patterns.	08 hrs	--	1	1	1
Chapter 4: GPU Architectures and Many-Core Systems.	07 hrs	--	1	1	1
Unit III					
Chapter 5: Parallelism in AI Systems.	05 hrs	--	--	1	1
Chapter 6: Scalable AI Systems.	05 hrs	--	--	1	1

Note

1. Each Question carries 20 marks and may consist of sub-questions.
2. Mixing of sub-questions from different chapters within a unit (only for Unit I and Unit II) is allowed in ISA I, II and ESA.
3. Answer 5 full questions of 20 marks each (two full questions from Unit I, II and one full question from Unit III) out of 8 questions in ESA.

Course Assessment Plan

Course Title :Parallel and GPU Computing					**Code : 26ECAC304**
Course outcomes (COs)	**Weightage in assessment**	**Assessment Methods**			
		ISA I	**ISA II**	**Lab Assessment**	**ESA (Theory)**
1. Explain the fundamentals of parallel computing, concurrency and parallelism, Flynn's taxonomy, and analyze speedup using Amdahl's Law and Gustafson's Law.	20%	✓		✓	✓
2. Analyze multicore architecture,shared and distributed memory systems, cache hierarchy, cache coherence,memory consistency,and synchronization mechanisms in NUMA systems.	20%	✓		✓	✓
3.Apply parallel programming models and frameworks such as pthreads, OpenMP, and Intel TBB for data/task parallelism, work partitioning, and scheduling.	20%		✓	✓	✓
4. Apply GPU architecture and the CUDA programming model to develop and optimize GPU kernels using the SIMT execution model and GPU memory hierarchy.	20%		✓	✓	✓
5. Optimize parallelism techniques in scalable AI models, including data/model/pipeline parallelism, GPU clusters, distributed training, Map Reduce, and performance profiling and optimization.	**20%**		✓	✓	✓
Weightage	100%	33%	33%		34%

Date: **Head, CSE(AI)**

Lab Activity and Rubrics:

Sl. No.	Experiments	CO	Blooms level	Timeline w.r.t COE	PI code	Hrs
1.	Analyze and Compare Parallel Computing Models for Matrix Multiplication	CO2	L3	1st & 2nd week	1.4.1	2
2.	Write multithreaded programs using parallel programming libraries.	CO1	L3	2nd & 3rd week	1.4.1/3.3.2	2
3.	Analyze synchronization mechanisms in parallel programs by implementing locks, semaphores, barriers, and evaluating their impact on performance and scalability.	CO3	L4	3rd & 4th week	3.3.2	2
4.	Cache coherence & NUMA memory access performance analysis.	CO3	L4	4th & 5th week	3.3.2	2
5.	Apply OpenMP directives to implement shared-memory parallelism for loop-level and task-level execution.	CO2	L4	5th & 6th week	2.1.2/3.3.2	2
6.	Implement parallel algorithmic patterns such as map, reduction, pipeline, and divide-and-conquer for efficient computation.	CO2	L4	6th & 7th week	2.1.2	2
7.	Write CUDA programs to understand GPU execution model, thread hierarchy, and kernel execution.	CO4	L4	7th & 8th week	3.3.2	2
8.	Implement parallel reduction using CUDA & analyze performance & memory optimization techniques.	CO3	L4	8th & 9th week	3.3.2	2
9.	Implement and evaluate GPU-based computations such as matrix multiplication or simple AI workload inference and compare CPU vs GPU performance.	CO4	L4	9th & 10th week	3.2.2	2
10.	Open ended experiments			11th & 12th week		

Parameter (evaluated for 10 M)	Excellent (8-10)	Good (4-7)	Fair (1-3)	PI	CO
Implementation of parallel and multithreaded programs (Expts 2,4,6,7)	Correct multicore/cache analysis; parallel program shows clear, measured speedup over sequential version; synchronization implemented correctly with no race conditions.	Parallel program implemented and runs correctly; speedup measured but not fully optimized; minor synchronization issues.	Unable to demonstrate measurable speedup. Synchronization errors (race conditions/deadlocks) present.	1.4.1/3.2.2	1, 2
OpenMP and parallel pattern implementation (Expts 1,5)	Correct use of OpenMP directives with optimized cache/NUMA-aware design; parallel patterns (map, reduction, pipeline, divide-and-conquer) implemented efficiently.	OpenMP directives used correctly; parallel patterns implemented with minor inefficiencies.	Unable to correctly apply OpenMP directives or implement parallel patterns.	2.1.2/3.2.2	2, 3
CUDA programming and GPU optimization (Expts 7,8)	CUDA kernels correctly use thread hierarchy and SIMT model; memory coalescing and shared memory optimization applied with measured performance gains.	CUDA kernels execute correctly with basic thread hierarchy usage; limited memory optimization.	Unable to execute CUDA kernel correctly or achieve expected GPU behavior.	3.3.2	4
GPU-based computation and scalable AI workload evaluation (Expts 9)	Correct CPU vs GPU performance comparison with clear throughput/latency analysis; scalable AI workload evaluated with sound resource-utilization conclusions.	Computation implemented correctly; performance comparison present but Analysis is limited.	Unable to implement GPU-based computation or evaluate scalability metrics.	3.2.2/5.2.1	4, 5

Chapter wise Plan

Course Code and Title: Parallel and GPU Computing	26ECAC304
Chapter Number and Title: 1. Introduction	Planned Hours: 7hrs

Learning Outcomes: -

At the end of the topic the student should be able to:

Topic Learning Outcomes	COs	BL	CA Code
1. Explain the motivation for parallel computing and the limitations of single-core processors.	CO1	L2	1.4
2. Differentiate between the concepts of concurrency and parallelism.	CO1	L2	1.4
3. Classify parallel computing systems using Flynn's taxonomy.	CO1	L2	1.4
4. Describe the different models of parallel computation.	CO1	L2	1.4
5. Apply Amdahl's Law to estimate the speedup achievable for a given parallel workload.	CO1	L3	1.4
6. Apply Gustafson's Law to analyze scaled speedup in parallel systems.	CO1	L3	1.4
7. Explain the evolution of multicore processors and the architecture of modern multicore CPUs.	CO1	L2	1.4
8. Analyze shared and distributed memory architectures, memory hierarchy and cache organization for performance.	CO1	L3	1.4

Lesson Schedule Class No. - Portion covered per hour
1. Motivation for parallel computing and limitations of single-core processors.
2. Concurrency and parallelism.
3. Flynn's taxonomy.
4. Models of parallel computation.
5. Amdahl's Law and Gustafson's Law.
6. Evolution of multicore processors and architecture of modern multicore CPUs.
7. Shared and distributed memory architectures, memory hierarchy, cache organization and performance considerations.

Review Questions

Sl. No.	Questions	TLOs	BL	PI Code
1.	Explain why single-core processors face fundamental performance limits and how parallel computing addresses these limits.	TLO1	L2	1.4.1
2.	Differentiate between concurrency and parallelism with suitable examples.	TLO2	L2	1.4.1
3.	Using Flynn's taxonomy, classify SISD, SIMD, MISD and MIMD systems, giving one example of each.	TLO3	L2	1.4.1
4.	Describe the common models of parallel computation and state one application of each.	TLO4	L2	1.4.1
5.	A scientific computing application spends 80% of its execution time in a parallelizable section. Using Amdahl's Law, calculate the maximum possible speedup on 8 cores and discuss the implication for scaling to more cores.	TLO5	L3	1.4.1

Sl. No.	Questions	TLOs	BL	PI Code
6.	Compare Amdahl's Law and Gustafson's Law, explaining when each is more appropriate for analyzing parallel system performance.	TLO6	L3	1.4.1
7.	Explain the evolution of multicore processors and describe the architecture of a modern multicore CPU.	TLO7	L2	1.4.1
8.	Discuss how shared memory and distributed memory architectures differ, and explain the role of cache hierarchy in multicore CPU performance.	TLO8	L2	1.4.1
9.	A multicore application shows diminishing returns beyond four cores. Analyze the possible cache-organization and memory-hierarchy reasons for this behaviour.	TLO8	L3	1.4.1

Chapter wise Plan

Title: Parallel and GPU Computing	**Course Code:24ECAC304**
Chapter Number and Title: 2. Memory Systems and Synchronization	Planned Hours: 8hrs

Learning Outcomes: -

At the end of the topic the student should be able to:

Topic Learning Outcomes	COs	BL	CA Code
1. Explain cache coherence protocols and their role in multicore systems.	CO2	L2	2.1
2. Describe memory consistency models and their impact on program correctness.	CO2	L2	2.1
3. Explain NUMA architecture and its memory access characteristics.	CO2	L2	2.1
4. Describe shared memory communication mechanisms used in parallel systems.	CO2	L2	2.1
5. Apply synchronization primitives such as locks and semaphores to coordinate parallel threads.	CO2	L3	2.1
6. Apply barriers and atomic operations to ensure correct parallel execution.	CO2	L3	2.1
7. Analyze the impact of false sharing and race conditions on parallel program performance.	CO2	L3	2.1
8. Analyze deadlocks and apply memory access optimization techniques to overcome performance challenges.	CO2	L3	2.1

Lesson Schedule Class No. - Portion covered per hour
1. Cache coherence protocols.
2. Memory consistency models.
3. NUMA architecture and memory access characteristics.
4. Shared memory communication mechanisms.
5. Locks and semaphores.
6. Barriers and atomic operations.
7. False sharing and race conditions.
8. Deadlocks, performance challenges and memory access optimization.

Review Questions

Sl. No.	**Questions**	**TLOs**	**BL**	**PI Code**
1.	Explain the need for cache coherence protocols in multicore systems and describe how they maintain data consistency across caches.	TLO1	L2	2.1.2
2.	Compare strict and relaxed memory consistency models and explain their effect on program correctness.	TLO2	L2	2.1.2
3.	Discuss how NUMA architecture affects memory access latency and describe strategies to optimize memory access in NUMA systems.	TLO3	L3	2.1.2
4.	Describe the shared memory communication mechanisms used in multicore systems.	TLO4	L2	2.1.2
5.	Compare locks, semaphores, and barriers as synchronization mechanisms, describing a scenario where each is most appropriate.	TLO5	L2	2.1.2

6.	Explain how atomic operations differ from lock-based synchronization, and discuss when each should be preferred.	TLO6	L3	2.1.2
7.	A multithreaded application performs poorly despite running on more cores. Explain how false sharing could be the cause and suggest ways to mitigate it.	TLO7	L3	2.1.2
8.	Explain race conditions with a suitable example and describe two techniques to eliminate them.	TLO7	L2	2.1.2
9.	A parallel banking application occasionally deadlocks when transferring funds between accounts. Explain the conditions necessary for deadlock and propose a strategy to prevent it.	TLO8	L3	2.1.2
10.	Discuss the major performance challenges in shared memory systems and describe memory access optimization techniques to address them.	TLO8	L3	2.1.2

Chapter wise Plan

Title: Parallel and GPU Computing	**Course Code:24ECAC304**
Chapter Number and Title: 3. Parallel Programming Models and Patterns	Planned Hours: 8hrs

Learning Outcomes: -

At the end of the topic the student should be able to:

Topic Learning Outcomes	COs	BL	CA Code
1. Explain thread-based programming models used in parallel computing.	CO3	L2	3.3
2. Differentiate between data parallelism and task parallelism.	CO3	L2	3.3
3. Apply shared-memory programming concepts using pthreads.	CO3	L3	3.3
4. Apply OpenMP directives to parallelize loop-level and task-level computations.	CO3	L3	3.3
5. Apply Intel TBB constructs to develop parallel applications.	CO3	L3	3.3
6. Analyze work partitioning strategies for balanced parallel execution.	CO3	L3	3.3
7. Analyze task scheduling techniques for parallel applications.	CO3	L3	3.3
8. Design parallel algorithms using patterns such as map, reduction, pipeline and divide-and-conquer.	CO3	L3	3.3

Lesson Schedule **Class No. - Portion covered per hour**
1. Thread-based programming models.
2. Data parallelism and task parallelism.
3. Shared-memory programming concepts and pthreads.
4. OpenMP directives for parallel execution.
5. Intel TBB.
6. Work partitioning strategies.
7. Task scheduling techniques.
8. Parallel algorithm design patterns: map, reduction, pipeline, and divide-and-conquer.

Review Questions

Sl. No.	**Questions**	**TLOs**	**BL**	**PI Code**
1.	Explain thread-based programming models and describe how threads are created and managed in a shared-memory system.	TLO1	L2	3.3.2
2.	Compare data parallelism and task parallelism, giving an example application suited to each.	TLO2	L2	3.3.2
3.	Illustrate with a simple program how a parallel task can be expressed using pthreads.	TLO3	L3	3.3.2
4.	Write and explain an OpenMP directive that parallelizes a loop-level computation, discussing potential race conditions.	TLO4	L3	3.3.2
5.	Compare pthreads, OpenMP, and Intel TBB as parallel programming frameworks, highlighting one advantage of each.	TLO5	L2	3.3.2

Sl. No.	Questions	TLOs	BL	PI Code
6.	Explain how work partitioning strategies affect load balancing and the overall performance of a parallel program.	TLO6	L3	3.3.2
7.	Describe static and dynamic task scheduling techniques and discuss the situations in which each is preferred.	TLO7	L3	3.3.2
8.	Describe the divide-and-conquer parallel pattern with an example, and explain how it differs from the pipeline pattern.	TLO8	L2	3.3.2
9.	A team needs to sum a very large array using multiple threads. Explain how the reduction and map patterns can be applied, and discuss the work-partitioning considerations involved.	TLO8	L3	3.3.2

Chapter wise Plan

Title: Parallel and GPU Computing	**Course Code:26ECAC304**
Chapter Number and Title: 4. GPU Architectures and Many-Core Systems	Planned Hours: 7hrs

Learning Outcomes: -

At the end of the topic the student should be able to:

Topic Learning Outcomes	COs	BL	CA Code
1. Explain GPU architectures and the organization of many-core systems.	CO4	L2	3.3
2. Describe the GPU execution model and the CUDA thread hierarchy.	CO4	L2	3.3
3. Explain the SIMT execution model and differentiate it from the SIMD model.	CO4	L2	3.3
4. Apply the CUDA programming model to develop basic GPU kernels.	CO4	L3	3.3
5. Explain the GPU memory hierarchy and the use of global, shared and register memory.	CO4	L2	3.3
6. Apply kernel execution and optimization techniques to improve GPU performance.	CO4	L3	3.3
7. Apply GPU-based parallel processing to accelerate data-intensive machine learning and deep learning workloads.	CO4	L3	3.3
8. Apply CUDA optimization techniques such as memory coalescing and thread scheduling to tune performance for AI workloads.	CO4	L3	3.3

Lesson Schedule **Class No. - Portion covered per hour**
1. GPU architectures and many-core systems.
2. GPU execution model and thread hierarchy.
3. SIMT execution model.
4. CUDA programming model.
5. GPU memory hierarchy, kernel execution and optimization techniques.
6. GPU-based parallel processing for data-intensive applications; acceleration of machine learning and deep learning workloads.
7. CUDA-based optimization: memory coalescing, thread scheduling, and performance tuning for AI workloads.

Review Questions

Sl. No.	Questions	TLOs	BL	PI Code
1.	Explain the architecture of a modern GPU and describe how it differs from a multicore CPU.	TLO1	L2	3.3.2
2.	Describe the GPU execution model and explain the CUDA thread hierarchy of grid, block and thread.	TLO2	L2	3.3.2
3.	Explain the SIMT execution model used in GPUs and how it differs from the SIMD model used in traditional vector processors.	TLO3	L2	3.3.2
4.	Describe the structure of a basic CUDA program, including kernel launch, thread hierarchy, and memory transfer between host and device.	TLO4	L2	3.3.2

Sl. No.	Questions	TLOs	BL	PI Code
5.	Compare the GPU memory hierarchy (global, shared, and registers) and explain when shared memory should be used over global memory.	TLO5	L2	3.3.2
6.	Explain kernel execution on a GPU and describe two optimization techniques that improve kernel throughput.	TLO6	L3	3.3.2
7.	Explain why GPU-based parallel processing is well suited to accelerating machine learning and deep learning workloads.	TLO7	L2	3.3.2
8.	A CUDA kernel shows poor performance due to uncoalesced global memory accesses. Explain the concept of memory coalescing and suggest how the kernel could be optimized.	TLO8	L3	3.3.2
9.	Explain how thread scheduling and occupancy influence the performance tuning of a CUDA kernel for an AI workload.	TLO8	L3	3.3.2

Chapter wise Plan

Title: Parallel and GPU Computing	**Course code:26ECAC304**
Chapter Number and Title: 5. Parallelism in AI Systems	Planned Hours: 5hrs

Learning Outcomes:-

At the end of the topic the student should be able to:

Topic Learning Outcomes	COs	BL	CA Code
1. Explain the characteristics of AI workloads in modern AI data centers.	CO5	L2	5.2
2. Explain data parallelism as applied to the training of AI models.	CO5	L2	5.2
3. Explain model parallelism and its use for very large neural networks.	CO5	L2	5.2
4. Explain pipeline parallelism and how it is combined with other parallelism strategies.	CO5	L2	5.2
5. Describe the organization of GPU clusters used for distributed training.	CO5	L2	5.2
6. Explain the fundamentals of distributed training of AI models.	CO5	L2	5.2
7. Analyze communication overheads and synchronization issues in distributed training.	CO5	L3	5.2
8. Analyze scalability, load balancing and performance trade-offs in distributed AI systems.	CO5	L3	5.2

Lesson Schedule Class No. - Portion covered per hour
1. AI workloads in modern AI data centers.
2. Data parallelism, model parallelism and pipeline parallelism.
3. GPU clusters and distributed training fundamentals.
4. Communication overheads and synchronization issues.
5. Scalability, load balancing and performance trade-offs.

Review Questions

Sl. No.	Questions	TLOs	BL	PI Code
1.	Describe the characteristics of AI workloads in modern data centers and explain why they require parallel hardware.	TLO1	L2	5.2.1
2.	Compare data parallelism and model parallelism in the context of training large AI models on GPU clusters.	TLO2	L2	5.2.1
3.	Explain model parallelism and describe a situation where it is preferred over data parallelism.	TLO3	L2	5.2.1
4.	Describe pipeline parallelism and explain how it can be combined with data parallelism to train very large models.	TLO4	L3	5.2.1
5.	Explain the organization of a GPU cluster and the role of interconnects in distributed training.	TLO5	L2	5.2.1

Sl. No.	Questions	TLOs	BL	PI Code
6.	Describe the fundamentals of distributed training, including parameter synchronization across workers.	TLO6	L2	5.2.1
7.	Explain why communication overhead becomes a bottleneck in distributed training and describe one technique to reduce it.	TLO7	L3	5.2.1
8.	Explain the synchronization issues that arise in synchronous and asynchronous distributed training.	TLO7	L3	5.2.1
9.	Discuss the scalability, load balancing and performance trade-offs involved in scaling a distributed training system to a larger number of GPUs.	TLO8	L3	5.2.1

Chapter wise Plan

Title: Parallel and GPU Computing	**Course code:26ECAC304**
Chapter Number and Title: 6. Scalable AI Systems	Planned Hours: 5hrs

Learning Outcomes: -

At the end of the topic the student should be able to:

Topic Learning Outcomes	COs	BL	CA Code
1. Explain the MapReduce programming model and its workflow.	CO5	L2	3.2
2. Apply the MapReduce model to a large-scale data processing problem.	CO5	L3	3.2
3. Explain how GPU-based acceleration is used for large-scale data processing.	CO5	L2	3.2
4. Describe the features of common distributed processing frameworks.	CO5	L2	3.2
5. Apply profiling techniques to measure the performance of parallel and GPU programs.	CO5	L3	3.2
6. Analyze compute-bound and memory-bound bottlenecks in AI workloads.	CO5	L3	3.2
7. Apply memory optimization techniques to improve the throughput of AI workloads.	CO5	L3	3.2
8. Apply kernel optimization techniques to accelerate deep learning training and inference.	CO5	L3	3.2

Lesson Schedule Class No. - Portion covered per hour
1. MapReduce programming model and workflow.
2. GPU-based acceleration for large-scale data processing.
3. Introduction to distributed processing frameworks.
4. Profiling and bottleneck analysis: compute-bound and memory-bound workloads.
5. Optimization techniques: memory and kernel optimization; case study of deep learning training and inference acceleration.

Review Questions

Sl. No.	Questions	TLOs	BL	PI Code
1.	Explain the MapReduce programming model with an example of how it processes large-scale data.	TLO1	L2	3.2.2
2.	Apply the MapReduce model to design a solution for counting word frequencies in a very large document collection.	TLO2	L3	3.2.2
3.	Explain how GPU-based acceleration improves large-scale data processing compared to traditional distributed CPU frameworks.	TLO3	L2	3.2.2
4.	Describe the features of a distributed processing framework and explain how it distributes work across nodes.	TLO4	L2	3.2.2
5.	Explain how profiling tools are used to measure the performance of a parallel or GPU program.	TLO5	L3	3.2.2
6.	Describe the difference between compute-bound and memory-bound	TLO6	L3	3.2.2

Sl. No.	Questions	TLOs	BL	PI Code
	workloads, and explain how profiling helps identify each.			
7.	A deep learning training pipeline is found to be memory-bound on the GPU. Suggest two memory optimization techniques that could improve its throughput.	TLO7	L3	3.2.2
8.	Explain the kernel optimization techniques that can be applied to accelerate deep learning training.	TLO8	L3	3.2.2
9.	Discuss, with reference to a case study, how profiling and optimization together accelerate deep learning inference.	TLO8	L3	3.2.2

Model Question Paper — ISA-1

Course Code: 26ECAC304	**Course Title: Parallel and GPU Computing**
Max. Marks: 40	**Duration: 75 Mins**

Note: *Answer any 2 full Questions. Each full Question carries 20 marks.*

Q.No	Questions	Marks	CO	BL	PO	PI Code
1a	A legacy image-processing application runs entirely on a single-core processor and takes several hours to process a batch of high-resolution images. (i) Explain the limitations of single-core processors that motivate the move to parallel computing. (ii) Using Flynn's taxonomy, classify SISD, SIMD, MISD and MIMD systems, and recommend a suitable classification for accelerating this application, with justification.	8	CO1	L3	1	1.4.1
1b	Explain the need for cache coherence protocols in multicore systems and describe how they maintain data consistency across per-core caches. Distinguish between strict and relaxed memory consistency models.	7	CO2	L2	2	2.1.2
1c	A data-analytics program parallelizes 60% of its total execution time. Using Amdahl's Law, compute the maximum theoretical speedup achievable on 6 processing cores.	5	CO1	L3	1	1.4.1
2a	Describe the evolution of multicore processors and the architecture of a modern multicore CPU. Differentiate between shared-memory and distributed-memory architectures, and explain the role of memory hierarchy and cache organization in determining multicore performance.	8	CO1	L2	1	1.4.1
2b	A multithreaded scientific-simulation program running on a NUMA machine shows unexpectedly poor scaling as more threads are added. Explain how NUMA architecture affects memory access latency, and describe how locks, semaphores and barriers can be used to correctly synchronize the threads without introducing excessive contention.	7	CO2	L3	2	2.1.2
2c	Define false sharing and explain, with a suitable example, how it degrades the performance of a multithreaded program even in the absence of any logical data dependency between threads.	5	CO2	L2	2	2.1.2
3a	A weather-simulation application currently parallelizes 70% of its workload. (i) Using Amdahl's Law, calculate the maximum speedup on 10 processors. (ii) If the problem size is scaled with the number of processors, use Gustafson's Law to estimate the speedup on the same 10 processors. Compare the two results.	8	CO1	L3	1	1.4.1
3b	A parallel banking application occasionally deadlocks when transferring funds concurrently between multiple accounts, and in other runs produces incorrect balances due to race conditions. Explain the four necessary conditions for deadlock and identify which of them arise in this scenario. (ii) Explain how atomic operations and correctly ordered	7	CO2	L4	2	2.1.2

Q.No	Questions	Marks	CO	BL	PO	PI Code
	locking can be used to eliminate both the race condition and the deadlock, and briefly discuss the performance trade-offs (e.g., false sharing, memory access optimization) of your solution.					
3c	For the banking application, explain how atomic operations and correctly ordered locking can be used to eliminate both the race condition and the deadlock. Briefly discuss the performance trade-offs of your solution, such as false sharing and memory-access optimization.	5	CO2	L4	2	2.1.2

Model Question Paper — ISA-2

Course Code: 26ECAC304	**Course Title: Parallel and GPU Computing**
Duration: 75 Mins	**Max. Marks: 40**

Note: *Answer any 2 full Questions. Each full Question carries 20 marks .*

Q.No	Questions	Marks	CO	BL	PO	PI Code
1a	Compare data parallelism and task parallelism, giving an example application suited to each. Explain thread-based, shared-memory programming concepts and illustrate how a simple parallel task can be expressed using pthreads.	8	CO3	L2	3	3.3.2
1b	Explain the SIMT execution model used in GPUs and how it differs from the SIMD model used in traditional vector processors. Illustrate, with a basic CUDA kernel, how the thread hierarchy (grid, block, thread) is used to process a large array.	7	CO4	L2	3	3.3.2
1c	Explain how work partitioning strategies affect load balancing and overall performance of a parallel program running on a multicore system.	5	CO3	L2	3	3.3.2
2a	A CUDA kernel performing matrix multiplication shows poor throughput due to uncoalesced global memory accesses. Explain the concept of memory coalescing and describe how the kernel could be re-designed to improve performance. Compare the roles of global, shared, and register memory in the GPU memory hierarchy.	8	CO4	L3	3	3.3.2
2b	Write and explain an OpenMP directive that parallelizes a loop-level computation, discussing potential race conditions. Explain how task scheduling techniques can be combined with the directive to balance load across threads.	7	CO3	L3	3	3.3.2
2c	Briefly explain why GPU-based parallel processing is well suited to accelerating machine learning and deep learning workloads compared to traditional CPU-only processing.	5	CO4	L2	3	3.3.2
3a	A team needs to design a parallel algorithm for summing a large array using multiple threads. (i) Explain how reduction and divide-and-conquer patterns could be applied, and discuss work-partitioning and task-scheduling considerations. (ii) Compare Pthreads, OpenMP, and Intel TBB as parallel programming frameworks, highlighting one advantage of each for implementing this algorithm.	8	CO3	L3	3	3.3.2
3b	A development team is building a CUDA program to accelerate an AI inference workload on a GPU cluster.Describe the structure of a basic CUDA program, including kernel launch, thread hierarchy, kernel execution, and host-device memory transfer.	7	CO4	L3	3	3.3.2
3c	For the CUDA-based AI inference workload, explain how memory coalescing and thread scheduling can be applied to tune GPU performance.	5	CO4	L3	3	3.3.2

Model Question Paper — ESA (End Semester Assessment)

Course Code: 26ECAC304	**Course Title: Parallel and GPU Computing**
Duration: 3 Hrs	**Max. Marks: 100**

Note: *Answer any TWO full Questions from Unit-I, any TWO full Questions from Unit-II, and any ONE full Question from Unit-III. Each full Question carries 20 marks.*

Q.No	Questions	Marks	CO	BL	PO	PI Code
	UNIT - I					
1a	A legacy image-processing application runs entirely on a single-core processor and takes several hours to process a batch of high-resolution images. (i) Explain the limitations of single-core processors that motivate the move to parallel computing. (ii) Using Flynn's taxonomy, classify SISD, SIMD, MISD and MIMD systems, and recommend a suitable classification for accelerating this application, with justification.	8	CO1	L3	1	1.4.1
1b	Explain the need for cache coherence protocols in multicore systems and describe how they maintain data consistency across per-core caches. Distinguish between strict and relaxed memory consistency models.	7	CO2	L2	2	2.1.2
1c	A data-analytics program parallelizes 60% of its total execution time. Using Amdahl's Law, compute the maximum theoretical speedup achievable on 6 processing cores.	5	CO1	L3	1	1.4.1
2a	Describe the evolution of multicore processors and the architecture of a modern multicore CPU. Differentiate between shared-memory and distributed-memory architectures, and explain the role of memory hierarchy and cache organization in determining multicore performance.	8	CO1	L2	1	1.4.1
2b	A multithreaded scientific-simulation program running on a NUMA machine shows unexpectedly poor scaling as more threads are added. Explain how NUMA architecture affects memory access latency, and describe how locks, semaphores and barriers can be used to correctly synchronize the threads without introducing excessive contention.	7	CO2	L3	2	2.1.2
2c	Define false sharing and explain, with a suitable example, how it degrades the performance of a multithreaded program even in the absence of any logical data dependency between threads.	5	CO2	L2	2	2.1.2
3a	A weather-simulation application currently parallelizes 70% of its workload. (i) Using Amdahl's Law, find the maximum speedup on 10 processors. (ii) If the problem size is scaled with the number of processors, use Gustafson's Law to estimate the speedup on the same 10 processors, assuming the same serial fraction. Discuss the difference between the two results.	8	CO1	L3	1	1.4.1
3b	A parallel banking application occasionally deadlocks when transferring funds concurrently between multiple accounts, and in other runs produces incorrect balances due to race conditions. Explain the four necessary conditions for deadlock and identify which of them	7	CO2	L4	2	2.1.2

Q.No	Questions	Marks	CO	BL	PO	PI Code
	arise in this scenario.					
3c	For the banking application, explain how atomic operations and correctly ordered locking can be used to eliminate both the race condition and the deadlock. Briefly discuss the performance trade-offs of your solution, such as false sharing and memory-access optimization.	5	CO2	L4	2	2.1.2
UNIT - II						
4a	Compare data parallelism and task parallelism, giving an example application suited to each. Explain thread-based, shared-memory programming concepts and illustrate how a simple parallel task can be expressed using pthreads.	8	CO3	L2	3	3.3.2
4b	Explain the SIMT execution model used in GPUs and how it differs from the SIMD model used in traditional vector processors. Illustrate, with a basic CUDA kernel, how the thread hierarchy (grid, block, thread) is used to process a large array.	7	CO4	L2	3	3.3.2
4c	Explain how work partitioning strategies affect load balancing and overall performance of a parallel program running on a multicore system.	5	CO3	L2	3	3.3.2
5a	A CUDA kernel performing matrix multiplication shows poor throughput due to uncoalesced global memory accesses. Explain the concept of memory coalescing and describe how the kernel could be re-designed to improve performance. Compare the roles of global, shared, and register memory in the GPU memory hierarchy.	8	CO4	L3	3	3.3.2
5b	Write and explain an OpenMP directive that parallelizes a loop-level computation, discussing potential race conditions. Explain how task scheduling techniques can be combined with the directive to balance load across threads.	7	CO3	L3	3	3.3.2
5c	Briefly explain why GPU-based parallel processing is well suited to accelerating machine learning and deep learning workloads compared to traditional CPU-only processing.	5	CO4	L2	3	3.3.2
6a	A team needs to design a parallel algorithm for summing a large array using multiple threads. (i) Explain how reduction and divide-and-conquer patterns could be applied, and discuss work-partitioning and task-scheduling considerations. (ii) Compare Pthreads, OpenMP, and Intel TBB as parallel programming frameworks, highlighting one advantage of each for implementing this algorithm.	8	CO3	L3	3	3.3.2
6b	A development team is building a CUDA program to accelerate an AI inference workload on a GPU cluster. Describe the structure of a basic CUDA program, including kernel launch, thread hierarchy, kernel execution, and host-device memory transfer.	7	CO4	L3	3	3.3.2
6c	For the CUDA-based AI inference workload, explain how memory coalescing and thread scheduling can be applied to tune performance.	5	CO4	L3	3	3.3.2
UNIT - III						

Q.No	Questions	Marks	CO	BL	PO	PI Code
7a	A deep-learning research team wants to train a very large neural network across multiple GPUs. Explain data parallelism, model parallelism, and pipeline parallelism, and describe how each can be used in distributed AI training.	8	CO5	L3	3	3.2.2
7b	In a multi-GPU training system, GPUs must frequently exchange model parameters, gradients, or intermediate results. Explain the communication overheads and synchronization issues that can arise during distributed training.	7	CO5	L3	3	3.2.2
7c	A distributed AI training system shows different GPUs completing their assigned work at different times. Explain the role of load balancing in distributed training and describe how poor load balancing can affect scalability and overall performance.	5	CO5	L3	3	3.2.2
8a	Explain the MapReduce programming model and its workflow with an example showing how a large dataset is processed using the Map and Reduce stages.	8	CO5	L3	3	3.2.2
8b	Compare GPU-based acceleration with traditional distributed CPU-based frameworks for large-scale data processing. Discuss the situations in which GPU acceleration can provide performance benefits.	7	CO5	L3	3	3.2.2
8c	A deep-learning training workload is found to spend most of its execution time waiting for data from GPU memory. How would you use profiling to identify this bottleneck, and what optimization techniques could be applied to improve performance?	5	CO5	L3	3	3.2.2